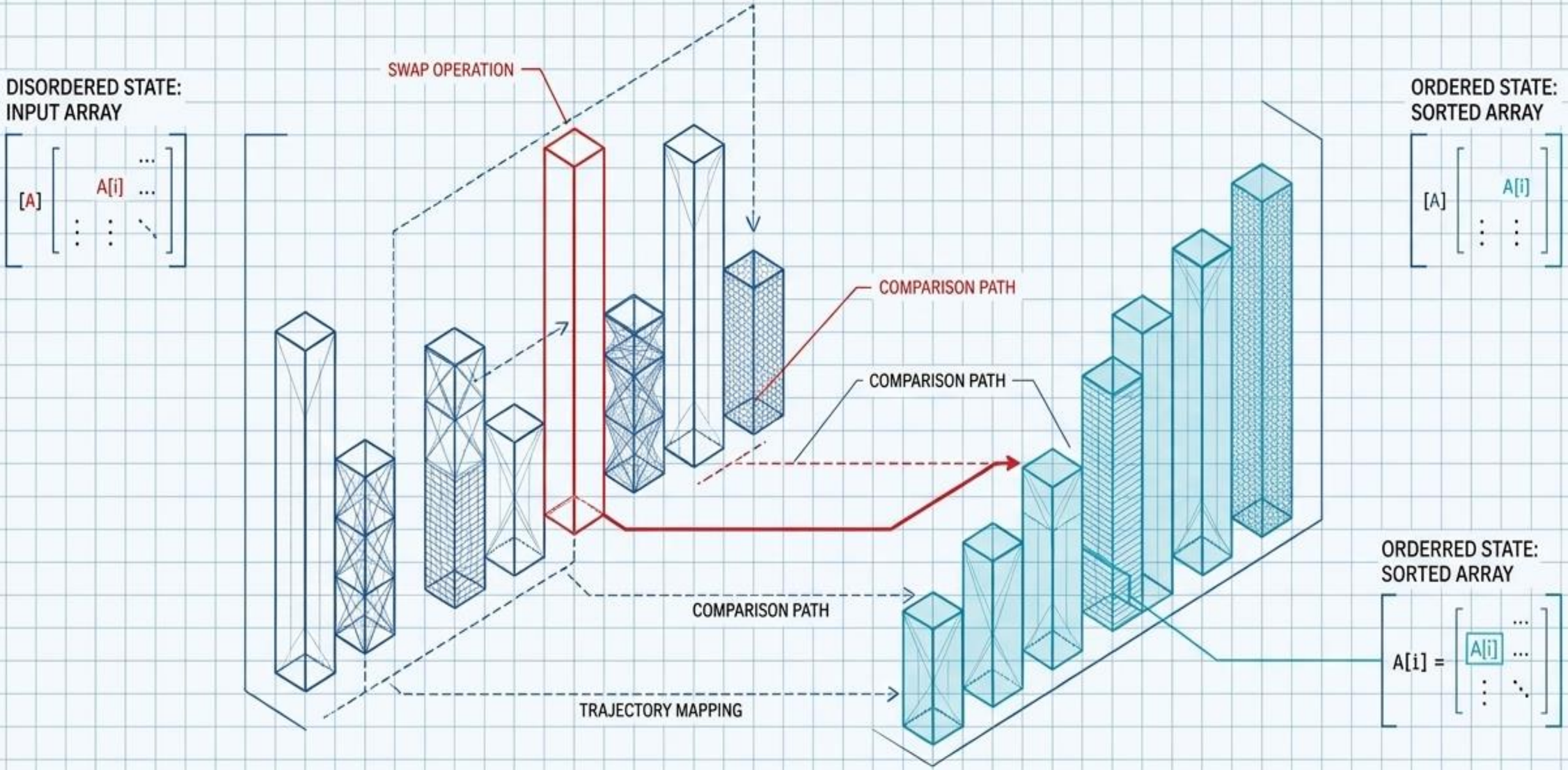
THE ARCHITECTURE OF ALGORITHMS

From Physical Intuition to Mathematical Rigor

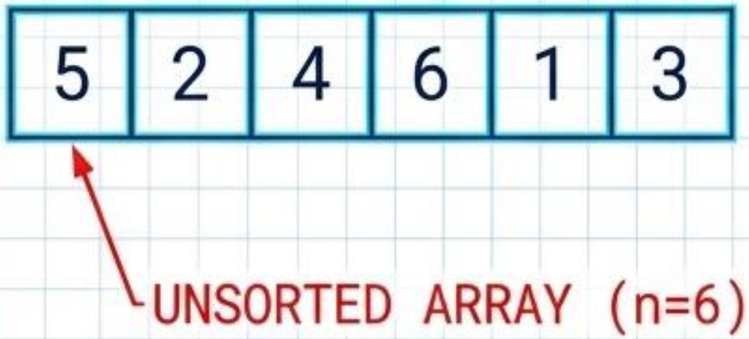


Defining the Core Objective: The Sorting Problem

Input

A sequence of n numbers (also known as keys).

<a1, a2, ..., an>

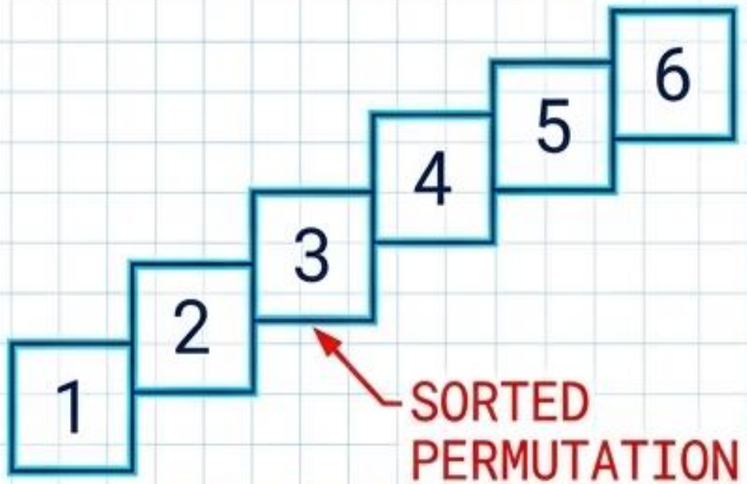


ALGORITHM
(The Transformation Engine)

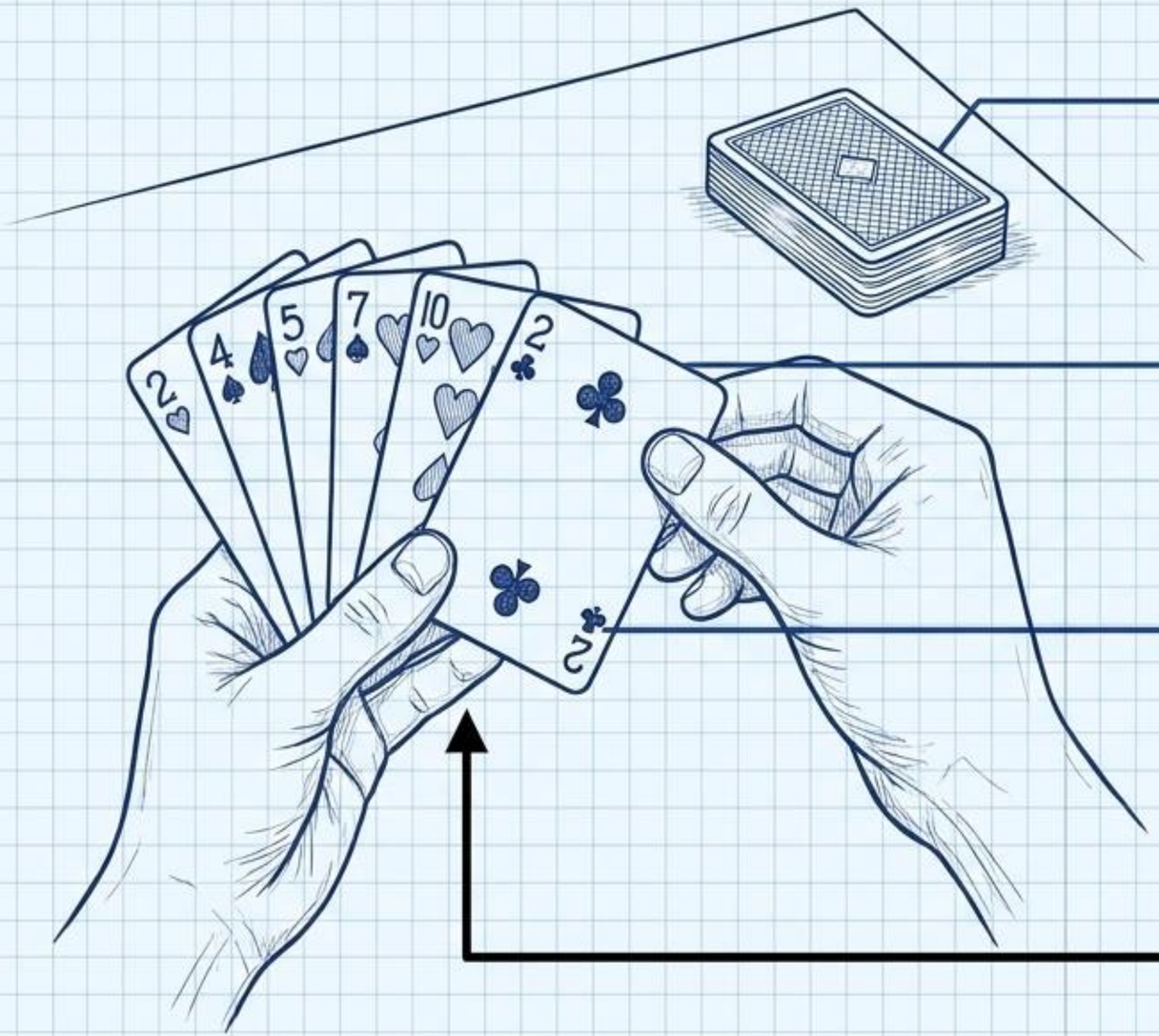
Output

A permutation (reordering) of the input sequence.

a'1 <= a'2 <= ... <= a'n



Paradigm 1: The Incremental Design of Insertion Sort



The Setup: Start with an empty left hand and cards face down on the table.

The Action: Remove one card from the table, insert it into the correct position in the left hand.

The Method: Compare the new card with cards already in the hand, scanning from right to left.

The Core Truth

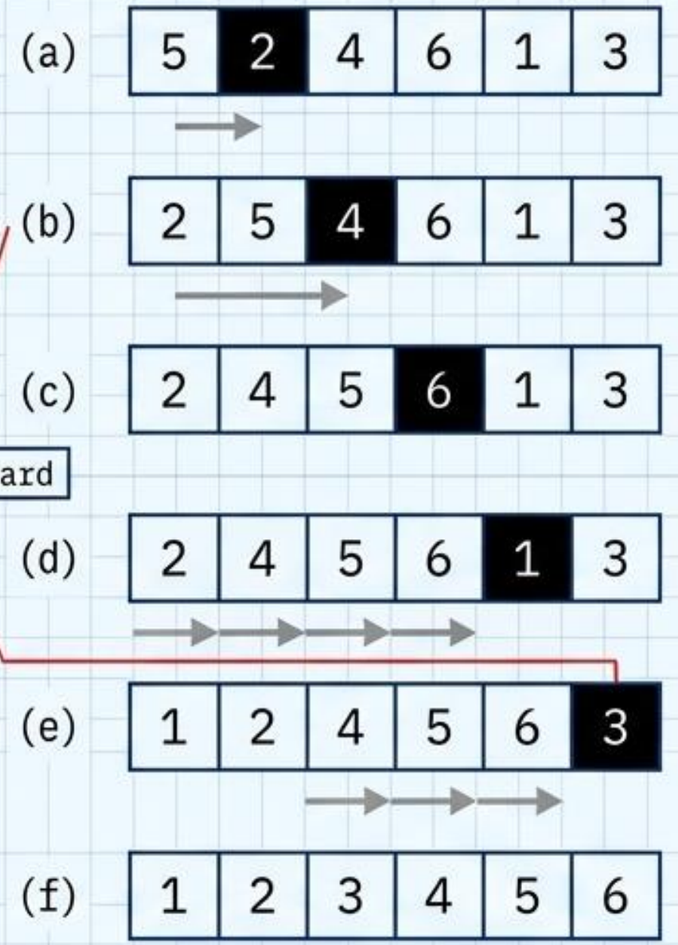
At all times, the cards held in the left hand are strictly sorted.

Translating Physical Action to Logical Code

The Logic (Pseudocode)

```
INSERTION-SORT(A)
1  for j = 2 to A.length
2      key = A[j]
3      // Insert A[j] into the sorted sequence A[1 .. j - 1].
4      i = j - 1
5      while i > 0 and A[i] > key
6          A[i + 1] = A[i]
7          i = i - 1
8      A[i + 1] = key
```

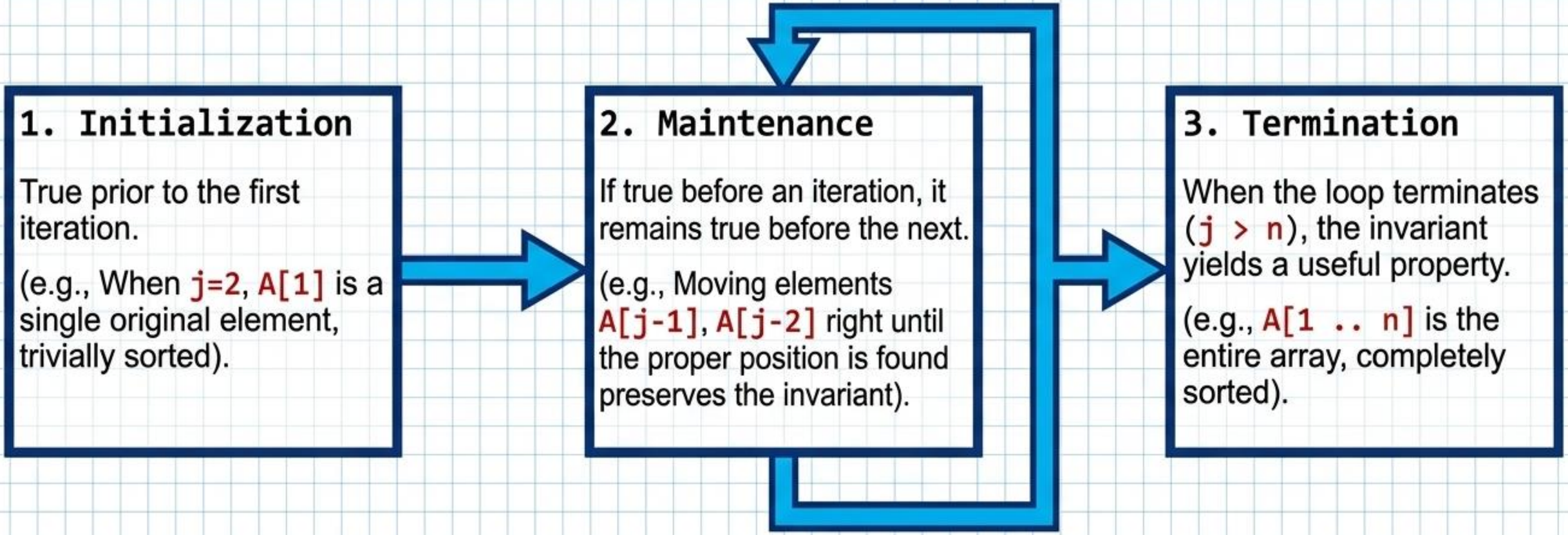
The Physical State



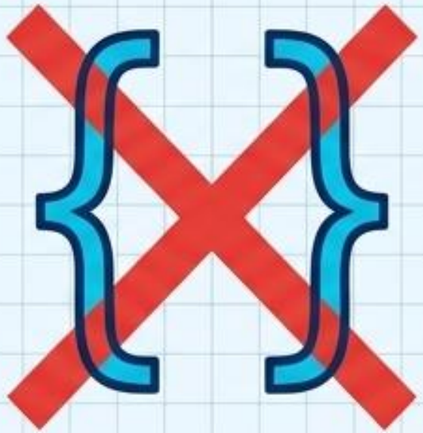
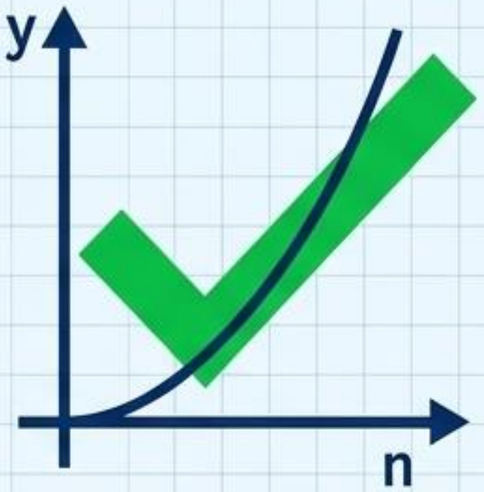
In-Place Sorting: Rearranges numbers within array A, with at most a constant number stored outside at any time.

Proving Correctness:
The Loop Invariant Architecture

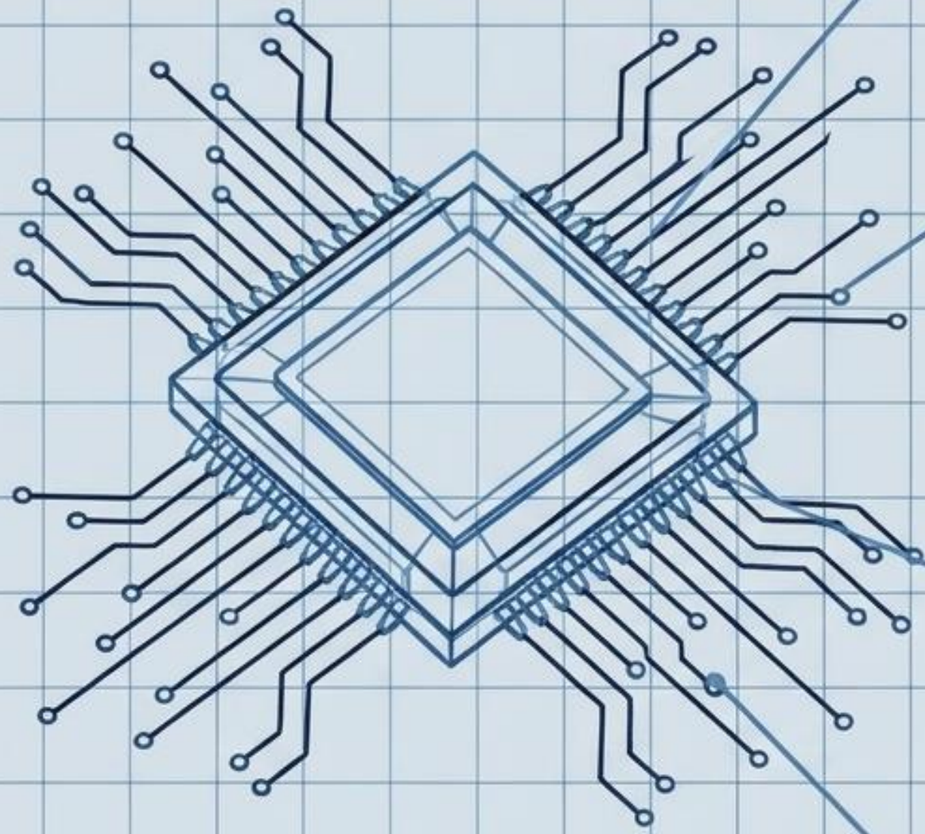
The Invariant Statement: At the start of each iteration of the `for` loop, the subarray `A[1 .. j-1]` consists of the elements originally in `A[1 .. j-1]`, but in **sorted order**.



The Measurement Dilemma: Finding an Objective Truth

Measuring Execution Time (Stopwatch)	Counting Statements Executed (Lines of Code)	Primitive Operations relative to Input Size
		
Flawed. Highly dependent on the specific hardware, CPU speed, and background OS tasks. The result is too variable.	**Flawed.** Varies wildly depending on the programming language chosen and the stylistic quirks of the individual programmer.	**Correct.** A mathematical, machine-independent model measuring absolute structural steps as the input volume (n) scales.

The Rules of Measurement: Constructing the RAM Model



Architecture

Random-Access Machine (RAM). Instructions execute sequentially, one after another. No concurrent operations.

Instruction Set

Contains common real-world instructions (add, subtract, load, store, branch). Each takes a strict constant amount of time.

Data Size Limit

Words hold a limit (e.g., integers represented by ***c* lg *n*** bits) so each word can hold the value of ***n*** to index the input elements, preventing arbitrarily large words that could compute in unrealistic constant time.

Exclusions

Ignores memory-hierarchy effects (caches, virtual memory) to keep the mathematical model tractable and universally applicable.

The Fundamental Equation of Cost Analysis

To evaluate an algorithm, associate a cost with each distinct statement, then multiply by its physical execution frequency.

The structural frequency dictated by sequence, logic, and loops (represented as N).

$$[\text{Total Cost}] = [\text{Statement Cost}] \times [\text{Execution Count}]$$

The absolute time the CPU takes to execute a single line of code once (represented as c1, c2, c3).

Let us first examine a few simple examples, and then return to the insertion sort example.

Algorithm 1: The Linear Sequence

Sequential execution scales simply. Every line executes exactly once per array element.

Code Structure	Mathematical Translation
`arr[0] = 0;`	`c1`
`arr[1] = 0;`	`c1`
`arr[2] = 0;`	`c1`
`...`	`...`
`arr[N-1] = 0;`	`c1`
	`c1 + c1 + ... + c1` = $c1 \times N$

Algorithm 2: The Single Loop

Condensing the code structurally hides an additional cost: the loop's condition check.

Code Structure	Mathematical Translation
`for(i=0; i<N; i++)`	c_2
`    arr[i] = 0;`	c_1
	Loop check: $(N+1) \times c_2$
	Loop body: $+ N \times c_1$
	$(c_2 + c_1) \times N + c_2$

Algorithm 3: Compounding Complexity via Nesting

Placing a loop inside a loop triggers multiplicative execution, resulting in quadratic growth.

Code Structure	Mathematical Translation
`sum = 0;`	$c1$
`for(i=0; i<N; i++)`	$c2$
`  for(j=0; j<N; j++)`	$c2$
`    sum += arr[i][j];`	$c3$

$$c1 + c2 \times (N+1) + c2 \times N \times (N+1) + c3 \times N^2$$

Analyzing Time: The Burden of Exact Cost

	Code	cost	times
1	`for j = 2 to A.length`	c1	n
2	`  key = A[j]`	c2	n - 1
3	`  i = j - 1`	c4	n - 1
4	`  while i > 0 and A[i] > key`	c5	$\sum_{j=2}^{n} t_j$
5	`  A[i + 1] = A[i]`	c6	$\sum_{j=2}^{n} (t_j - 1)$
6	`  i = i - 1`	c7	$\sum_{j=2}^{n} (t_j - 1)$
7	`  A[i + 1] = key`	c8	n - 1

$$T(n) = c1*n + c2(n-1) + c4(n-1) + c5\sum_{j=2}^{n} t_j + c6\sum_{j=2}^{n} (t_j-1) + c7\sum_{j=2}^{n} (t_j-1) + c8(n-1)$$

Calculating exact cost yields a hyper-specific, unwieldy mathematical polynomial. It is precise, but practically useless for rapid architectural decisions.

Algorithm Diagnostics: Best-Case vs. Worst-Case Limits

Algorithm Code Anatomy

Best-Case (Array already sorted)

$t_j = 1$. The inner while loop immediately exits.

$$T(n) = an + b$$

A Linear Function

Worst-Case (Reverse sorted)

$t_j = j$. The inner loop must compare every existing element.

$$T(n) = an^2 + bn + c$$

A Quadratic Function

The Worst-Case Imperative

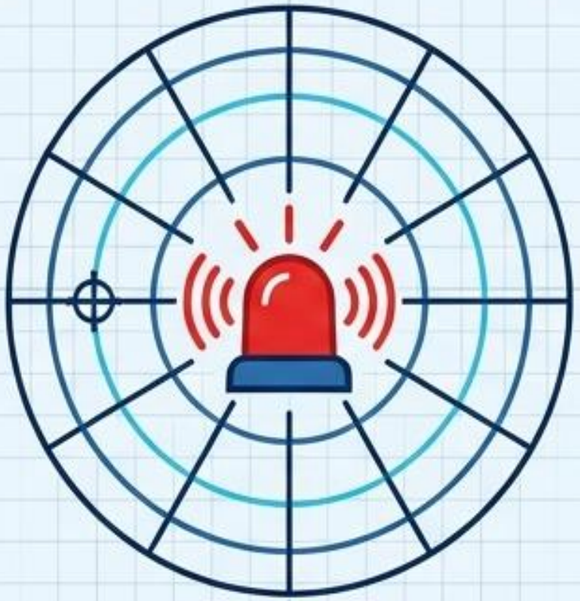
Why do computer scientists concentrate primarily on finding the longest possible running time for any input of size *n*?

The Absolute Guarantee



It provides an upper bound. Knowing the worst-case means the algorithm will never take any longer. There is no need to make educated guesses or hope for the best.

Frequency of Failure



The worst-case scenario occurs fairly often in practice. For example, in a database search, looking for a piece of information that is not present triggers the worst-case behavior.

The Average is the Worst

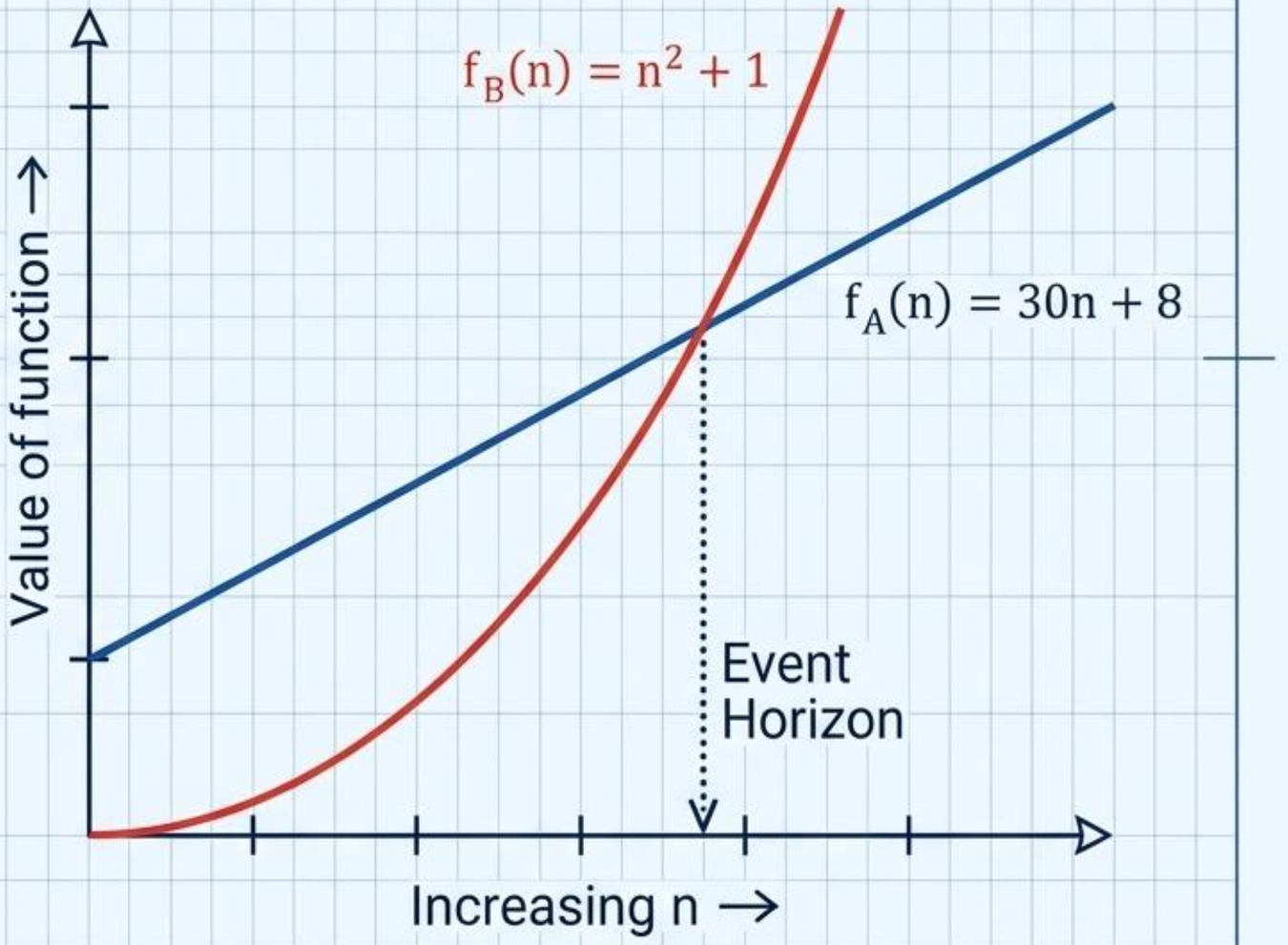


The "average case" is often roughly as bad as the worst case. When sorting randomly, we expect to check half the subarray ($t_j \approx j/2$), which still results in a **quadratic function** of the input size.

The Event Horizon: Visualizing Orders of Growth

On a graph, as you go to the right, a **faster growing function eventually** becomes **larger...**

Regardless of massive constant multipliers (like the 30 in 30n), a higher-order function (like n^2) will always eventually become infinitely larger "in the limit" as data scales.



Asymptotic Analysis: Dropping the Noise

The Elephant and Goldfish Principle

Imagine calculating the total weight of an elephant holding a goldfish.

Total = Weight of Elephant + Weight of Goldfish.

Because the goldfish's weight is infinitesimalally small compared to the elephant, the Total is approximately just the Weight of the Elephant.

Mathematical Application:

$$n^4 + 100n^2 + 10n + 50 \Rightarrow \sim n^4$$

The low order terms in a function are relatively insignificant for large n.

The Simplification Pipeline: Defining Order of Growth

Step 1: The Raw Formula

$an^2 + bn + c$

Step 2: Isolate Leading Term

$an^2 + ~~bn + c~~$

We consider only the leading term. Lower-order terms become insignificant for large values of n.

Step 3: Ignore Constants

$~~a~~n^2$

We ignore the constant coefficient. Constant factors matter far less than the rate of growth for large inputs.

Step 4: The Asymptotic Result

$\Theta(n^2)$

Pronounced “theta of n-squared”. This is the formal Order of Growth.